ITSAR2 313 – SYSTEM INTEGRATION AND ARCHICTECTURE 2
LABORATORY REPORT: EXERCISE 2 MICROSERVICES EDGE CASE TESTING (CURL-BASED)

Delicano, Rovin Destajo; Herman, Angel Mae Arnaiz; Olmedo, Jirah Canillo; Prado, Jhona Mae Octavio; Sudayon, Karyl Ann Delmita
Information Technology, College of Computer Studies
Carlos Hilado Memorial State University

Abstract

This laboratory exercise is an extension to the Student Course System developed in laboratory 1 by applying systematic edge case testing across all three microservices using curl. The objective was to verify that each service returns correct HTTP status codes and consistent JSON error responses when subjected to invalid input, missing resources, duplicate requests, and dependency failures. All tests were conducted using curl with the -i flag from the command line to capture response headers and status codes. Results confirmed that all required edge case behaviors 400, 404, 409, 503, 504 were correctly implemented and observable through standard HTTP tooling.

I. INTRODUCTION

II. METHODOLOGY / IMPLEMENTATION

The three services were started simultaneously using the start.ps1 PowerShell script, which launched each Flask application as a background job on each designated ports: Student Service on 8001, Course Service on 8002, and Enrollment Service on 8003. A Python HTTP server served as a frontend on port 5500. Test were executed using Windows Command Prompt using curl.exe.

Each test was issued as a single curl command with the -I flag to include response headers in the output. JSON request bodies were passed using the -d flag with escaped double quotes, and the Content-Type header was set explicitly using -H. For dependency failure tests, individual services were stopped using Stop-Job in PowerShell and then restarted after the test to restore the system state. All tests were captured through screenshot.

The test suite covered five categories of edge cases as required by the laboratory specification: invalid or missing input (400 Bad Request), non-existent resources (404 Not Found), duplicate enrollment attempts (409 Conflict), dependency service failures (503 Service Unavailable), and dependency timeouts (504 Gateway Timeout). A total of 29 test cases were executed across the three services.

III. EXPERIMENTAL FINDINGS / OBSERVATIONS

The following test cases document each curl command, the expected HTTP status code, and the expected JSON response body. All services must be running before executing tests 1 through 28. Test 29 requires the Student Service to be stopped first.

A. Student Service (port 8001)

Test	**1. Create a valid student**
curl command	`curl.exe -i -X POST "http://localhost:8001/students" -H "Content-Type: application/json" -d "{\"name\":\"Janel Perez\",\"email\":\"janel.perez@chmsu.edu.ph\"}"`
Expected	`201 Created — {"id": 1, "message": "created"}`
Note	First successful student creation. ID will increment on subsequent calls.

```
D:\Projects\microservices>curl -i -X POST "http://localhost:8001/students" -H "Content-Type: application/json" -d "{\"name\":\"Janel Perez\",\"email\":\"janel.perez@chmsu.edu.ph\"}"
HTTP/1.1 201 CREATED
Server: Werkzeug/3.1.6 Python/3.14.3
Date: Wed, 11 Mar 2026 13:13:31 GMT
Content-Type: application/json
Content-Length: 38
Access-Control-Allow-Origin: *
Connection: close

{
  "id": 1,
  "message": "created"
}
```

Test	2. Create a second valid student
curl command	curl.exe -i -X POST "http://localhost:8001/students" -H "Content-Type: application/json" -d "{\"name\":\"John Doe \",\"email\":\"john.doe@chmsu.edu.ph\"}"
Expected	201 Created — {"id": 2, "message": "created"}

```
D:\Projects\microservices>curl -i -X POST "http://localhost:8001/students" -H "Content-Type: application/json" -d "{\"name\":\"John Doe\",\"email\":\"john.doe@chmsu.edu.ph\"}"
HTTP/1.1 201 CREATED
Server: Werkzeug/3.1.6 Python/3.14.3
Date: Wed, 11 Mar 2026 13:16:14 GMT
Content-Type: application/json
Content-Length: 38
Access-Control-Allow-Origin: *
Connection: close

{
  "id": 2,
  "message": "created"
}
```

Test	3. Get all students
curl command	curl.exe -i "http://localhost:8001/students"
Expected	200 OK — [{"id": 1, "name": "Janel Perez", "email": "janel.perez@chmsu.edu.ph"}, ...]

```
D:\Projects\microservices>curl -i "http://localhost:8001/students"
HTTP/1.1 200 OK
Server: Werkzeug/3.1.6 Python/3.14.3
Date: Wed, 11 Mar 2026 13:16:49 GMT
Content-Type: application/json
Content-Length: 175
Access-Control-Allow-Origin: *
Connection: close

[
  {
    "email": "janel.perez@chmsu.edu.ph",
    "id": 1,
    "name": "Janel Perez"
  },
  {
    "email": "john.doe@chmsu.edu.ph",
    "id": 2,
    "name": "John Doe"
  }
]
```

Test	4. Get student by ID
curl command	curl.exe -i "http://localhost:8001/students/1"
Expected	200 OK — {"id": 1, "name": "Janel Perez", "email": "janel.perez@chmsu.edu.ph"}

```
D:\Projects\microservices>curl -i "http://localhost:8001/students/1"
HTTP/1.1 200 OK
Server: Werkzeug/3.1.6 Python/3.14.3
Date: Wed, 11 Mar 2026 13:17:21 GMT
Content-Type: application/json
Content-Length: 78
Access-Control-Allow-Origin: *
Connection: close

{
  "email": "janel.perez@chmsu.edu.ph",
  "id": 1,
  "name": "Janel Perez"
}
```

Test	5. [EDGE CASE] Get student — not found
curl command	`curl.exe -i "http://localhost:8001/students/999"`
Expected	`404 Not Found — {"error": "NOT_FOUND", "message": "Student with id 999 does not exist"}`
Note	Verifies that requesting a non-existent ID return 404 with a descriptive error body.

```
D:\Projects\microservices>curl -i "http://localhost:8001/students/999"
HTTP/1.1 404 NOT FOUND
Server: Werkzeug/3.1.6 Python/3.14.3
Date: Wed, 11 Mar 2026 13:17:54 GMT
Content-Type: application/json
Content-Length: 78
Access-Control-Allow-Origin: *
Connection: close

{
  "error": "NOT_FOUND",
  "message": "Student with id 999 does not exist"
}
```

Test	6. [EDGE CASE] Create student — missing email field
curl command	`curl.exe -i -X POST "http://localhost:8001/students" -H "Content-Type: application/json" -d "{\"name\":\"No Email\"}"`
Expected	`400 Bad Request — {"error": "MISSING_FIELDS", "message": "Both name and email are required"}`
Note	Verifies field validation — a request missing the email key is rejected before any database operation.

```
D:\Projects\microservices>curl -i -X POST "http://localhost:8001/students" -H "Content-Type: application/json" -d "{\"name\":\"No Email\"}"
HTTP/1.1 400 BAD REQUEST
Server: Werkzeug/3.1.6 Python/3.14.3
Date: Wed, 11 Mar 2026 13:18:47 GMT
Content-Type: application/json
Content-Length: 81
Access-Control-Allow-Origin: *
Connection: close

{
  "error": "MISSING_FIELDS",
  "message": "Both name and email are required"
}
```

Test	7. [EDGE CASE] Create student — empty body
curl command	`curl.exe -i -X POST "http://localhost:8001/students" -H "Content-Type: application/json" -d "{}"`
Expected	`400 Bad Request — {"error": "MISSING_FIELDS", "message": "Both name and email are required"}`

```
D:\Projects\microservices>curl -i -X POST "http://localhost:8001/students" -H "Content-Type: application/json" -d "{}"
HTTP/1.1 400 BAD REQUEST
Server: Werkzeug/3.1.6 Python/3.14.3
Date: Wed, 11 Mar 2026 13:19:52 GMT
Content-Type: application/json
Content-Length: 81
Access-Control-Allow-Origin: *
Connection: close

{
  "error": "MISSING_FIELDS",
  "message": "Both name and email are required"
}
```

Test	8. [EDGE CASE] Create student — duplicate email
curl command	`curl.exe -i -X POST "http://localhost:8001/students" -H "Content-Type: application/json" -d "{\"name\":\"Janel Perez\",\"email\":\"janel.perez@chmsu.edu.ph\"}"`
Expected	`409 Conflict — {"error": "DUPLICATE_EMAIL", "message": "A student with email 'janel.perez@chmsu.edu.ph' already exists"}`
Note	The email uniqueness constraint is enforced at the application layer before the database write.

```
D:\Projects\microservices>curl -i -X POST "http://localhost:8001/students" -H "Content-Type: application/json" -d "{\"name\":\"Janel Perez\",\"email\":\"janel.perez@chmsu.edu.ph\"}"
HTTP/1.1 409 CONFLICT
Server: Werkzeug/3.1.6 Python/3.14.3
Date: Wed, 11 Mar 2026 13:21:04 GMT
Content-Type: application/json
Content-Length: 112
Access-Control-Allow-Origin: *
Connection: close

{
  "error": "DUPLICATE_EMAIL",
  "message": "A student with email 'janel.perez@chmsu.edu.ph' already exists"
}
```

Test	9. Delete student by ID
curl command	`curl.exe -i -X DELETE "http://localhost:8001/students/2"`
Expected	`200 OK — {"id": 2, "message": "deleted"}`

```
D:\Projects\microservices>curl -i -X DELETE "http://localhost:8001/students/2"
HTTP/1.1 200 OK
Server: Werkzeug/3.1.6 Python/3.14.3
Date: Wed, 11 Mar 2026 13:21:40 GMT
Content-Type: application/json
Content-Length: 38
Access-Control-Allow-Origin: *
Connection: close

{
  "id": 2,
  "message": "deleted"
}
```

Test	10. [EDGE CASE] Delete student — not found
curl command	`curl.exe -i -X DELETE "http://localhost:8001/students/999"`
Expected	`404 Not Found — {"error": "NOT_FOUND", "message": "Student with id 999 does not exist"}`

```
D:\Projects\microservices>curl -i -X DELETE "http://localhost:8001/students/999"
HTTP/1.1 404 NOT FOUND
Server: Werkzeug/3.1.6 Python/3.14.3
Date: Wed, 11 Mar 2026 13:22:13 GMT
Content-Type: application/json
Content-Length: 78
Access-Control-Allow-Origin: *
Connection: close

{
  "error": "NOT_FOUND",
  "message": "Student with id 999 does not exist"
}
```

B. Course Service (port 8002)

Test	11. Create a valid course
curl command	curl.exe -i -X POST "http://localhost:8002/courses" -H "Content-Type: application/json" -d "{\"title\":\"Computer Programming 101\",\"credits\":5}"
Expected	201 Created — {"id": 1, "message": "created"}

```
D:\Projects\microservices>curl -i -X POST "http://localhost:8002/courses" -H "Content-Type: application/json" -d "{\"title\":\"Computer Programming 101\",\"credits\":5}"
HTTP/1.1 201 CREATED
Server: Werkzeug/3.1.6 Python/3.14.3
Date: Wed, 11 Mar 2026 13:23:48 GMT
Content-Type: application/json
Content-Length: 38
Access-Control-Allow-Origin: *
Connection: close

{
  "id": 1,
  "message": "created"
}
```

Test	12. Create a second valid course
curl command	curl.exe -i -X POST "http://localhost:8002/courses" -H "Content-Type: application/json" -d "{\"title\":\"Accounting 101\",\"credits\":3}"
Expected	201 Created — {"id": 2, "message": "created"}

```
D:\Projects\microservices>curl -i -X POST "http://localhost:8002/courses" -H "Content-Type: application/json" -d "{\"title\":\"Accounting 101\",\"credits\":3}"
HTTP/1.1 201 CREATED
Server: Werkzeug/3.1.6 Python/3.14.3
Date: Wed, 11 Mar 2026 13:24:31 GMT
Content-Type: application/json
Content-Length: 38
Access-Control-Allow-Origin: *
Connection: close

{
  "id": 2,
  "message": "created"
}
```

Test	13. Get all courses
curl command	curl.exe -i "http://localhost:8002/courses"
Expected	200 OK — [{"id": 1, "title": "Math 101", "credits": 3}, ...]

```
D:\Projects\microservices>curl -i -X POST "http://localhost:8002/courses" -H "Content-Type: application/json" -d "{\"title\":\"Accounting 101\",\"credits\":3}"
HTTP/1.1 201 CREATED
Server: Werkzeug/3.1.6 Python/3.14.3
Date: Wed, 11 Mar 2026 13:24:31 GMT
Content-Type: application/json
Content-Length: 38
Access-Control-Allow-Origin: *
Connection: close

{
  "id": 2,
  "message": "created"
}
```

Test	14. Get course by ID
curl command	curl.exe -i "http://localhost:8002/courses/1"
Expected	200 OK — {"id": 1, "title": "Math 101", "credits": 3}

```
D:\Projects\microservices>curl -i "http://localhost:8002/courses/1"
HTTP/1.1 200 OK
Server: Werkzeug/3.1.6 Python/3.14.3
Date: Wed, 11 Mar 2026 13:25:27 GMT
Content-Type: application/json
Content-Length: 69
Access-Control-Allow-Origin: *
Connection: close

{
  "credits": 5,
  "id": 1,
  "title": "Computer Programming 101"
}
```

Test	15. [EDGE CASE] Get course — not found
curl command	`curl.exe -i "http://localhost:8002/courses/999"`
Expected	404 Not Found — {"error": "NOT_FOUND", "message": "Course with id 999 does not exist"}

```
D:\Projects\microservices>curl -i "http://localhost:8002/courses/1"
HTTP/1.1 200 OK
Server: Werkzeug/3.1.6 Python/3.14.3
Date: Wed, 11 Mar 2026 13:25:27 GMT
Content-Type: application/json
Content-Length: 69
Access-Control-Allow-Origin: *
Connection: close

{
  "credits": 5,
  "id": 1,
  "title": "Computer Programming 101"
}
```

Test	16. [EDGE CASE] Create course — missing credits
curl command	`curl.exe -i -X POST "http://localhost:8002/courses" -H "Content-Type: application/json" -d "{\"title\":\"No Credits\"}"`
Expected	400 Bad Request — {"error": "MISSING_FIELDS", "message": "Both title and credits are required"}

```
D:\Projects\microservices>curl -i -X POST "http://localhost:8002/courses" -H "Content-Type: application/json" -d "{\"title\":\"No Credits\"}"
HTTP/1.1 400 BAD REQUEST
Server: Werkzeug/3.1.6 Python/3.14.3
Date: Wed, 11 Mar 2026 13:26:22 GMT
Content-Type: application/json
Content-Length: 84
Access-Control-Allow-Origin: *
Connection: close

{
  "error": "MISSING_FIELDS",
  "message": "Both title and credits are required"
}
```

Test	17. [EDGE CASE] Create course — invalid credits (zero)
curl command	`curl.exe -i -X POST "http://localhost:8002/courses" -H "Content-Type: application/json" -d "{\"title\":\"Bad Credits\",\"credits\":0}"`
Expected	400 Bad Request — {"error": "INVALID_INPUT", "message": "Credits must be a positive integer"}
Note	Credits of 0 or negative values are rejected. Only positive integers are accepted.

```
D:\Projects\microservices>curl -i -X POST "http://localhost:8002/courses" -H "Content-Type: application/json" -d "{\"title\":\"Bad Credits\",\"credits\":0}"
HTTP/1.1 400 BAD REQUEST
Server: Werkzeug/3.1.6 Python/3.14.3
Date: Wed, 11 Mar 2026 13:27:31 GMT
Content-Type: application/json
Content-Length: 82
Access-Control-Allow-Origin: *
Connection: close

{
  "error": "INVALID_INPUT",
  "message": "Credits must be a positive integer"
}
```

Test	18. [EDGE CASE] Create course — empty body
curl command	`curl.exe -i -X POST "http://localhost:8002/courses" -H "Content-Type: application/json" -d "{}"`
Expected	`400 Bad Request — {"error": "MISSING_FIELDS", "message": "Both title and credits are required"}`

```
D:\Projects\microservices>curl -i -X POST "http://localhost:8002/courses" -H "Content-Type: application/json" -d "{}"
HTTP/1.1 400 BAD REQUEST
Server: Werkzeug/3.1.6 Python/3.14.3
Date: Wed, 11 Mar 2026 13:28:16 GMT
Content-Type: application/json
Content-Length: 84
Access-Control-Allow-Origin: *
Connection: close

{
  "error": "MISSING_FIELDS",
  "message": "Both title and credits are required"
}
```

Test	19. [EDGE CASE] Delete course — not found
curl command	`curl.exe -i -X DELETE "http://localhost:8002/courses/999"`
Expected	`404 Not Found — {"error": "NOT_FOUND", "message": "Course with id 999 does not exist"}`

```
D:\Projects\microservices>curl -i -X DELETE "http://localhost:8002/courses/999"
HTTP/1.1 404 NOT FOUND
Server: Werkzeug/3.1.6 Python/3.14.3
Date: Wed, 11 Mar 2026 13:30:29 GMT
Content-Type: application/json
Content-Length: 77
Access-Control-Allow-Origin: *
Connection: close

{
  "error": "NOT_FOUND",
  "message": "Course with id 999 does not exist"
}
```

C. Enrollment Service (port 8003)

Test	20. Create a valid enrollment
curl command	`curl.exe -i -X POST "http://localhost:8003/enrollments" -H "Content-Type: application/json" -d "{\"student_id\":1,\"course_id\":1}"`
Expected	`201 Created — {"id": 1, "message": "created"}`
Note	Requires student ID 1 and course ID 1 to exist. Run tests 1 and 11 first.

```
D:\Projects\microservices>curl -i -X POST "http://localhost:8003/enrollments" -H "Content-Type: application/json" -d "{\"student_id\":1,\"course_id\":1}"
HTTP/1.1 201 CREATED
Server: Werkzeug/3.1.6 Python/3.14.3
Date: Wed, 11 Mar 2026 13:32:23 GMT
Content-Type: application/json
Content-Length: 38
Access-Control-Allow-Origin: *
Connection: close

{
  "id": 1,
  "message": "created"
}
```

Test	21. Create a second valid enrollment
curl command	`curl.exe -i -X POST "http://localhost:8003/enrollments" -H "Content-Type: application/json" -d "{\"student_id\":1,\"course_id\":2}"`
Expected	`201 Created — {"id": 2, "message": "created"}`

```
D:\Projects\microservices>curl -i -X POST "http://localhost:8003/enrollments" -H "Content-Type: application/json" -d "{\"student_id\":1,\"course_id\":1}"
HTTP/1.1 201 CREATED
Server: Werkzeug/3.1.6 Python/3.14.3
Date: Wed, 11 Mar 2026 13:32:23 GMT
Content-Type: application/json
Content-Length: 38
Access-Control-Allow-Origin: *
Connection: close

{
  "id": 1,
  "message": "created"
}
```

Test	22. Get all enrollments
curl command	`curl.exe -i "http://localhost:8003/enrollments"`
Expected	`200 OK — [{"id": 1, "student": {...}, "course": {...}}, ...]`
Note	Each enrollment record is enriched with the full student and course objects via inter-service HTTP calls.

```
D:\Projects\microservices>curl -i "http://localhost:8003/enrollments"
HTTP/1.1 200 OK
Server: Werkzeug/3.1.6 Python/3.14.3
Date: Wed, 11 Mar 2026 13:34:16 GMT
Content-Type: application/json
Content-Length: 455
Access-Control-Allow-Origin: *
Connection: close

[
  {
    "course": {
      "credits": 5,
      "id": 1,
      "title": "Computer Programming 101"
    },
    "id": 1,
    "student": {
      "email": "janel.perez@chmsu.edu.ph",
      "id": 1,
      "name": "Janel Perez"
    }
  },
  {
    "course": {
      "credits": 3,
      "id": 2,
      "title": "Accounting 101"
    },
    "id": 2,
    "student": {
      "email": "janel.perez@chmsu.edu.ph",
      "id": 1,
      "name": "Janel Perez"
    }
  }
]
```

Test	23. Get enrollment by ID
curl command	`curl.exe -i "http://localhost:8003/enrollments/1"`
Expected	200 OK — {"id": 1, "student": {"id": 1, "name": "Alice Santos", ...}, "course": {...}}

```
D:\Projects\microservices>curl -i "http://localhost:8003/enrollments"
HTTP/1.1 200 OK
Server: Werkzeug/3.1.6 Python/3.14.3
Date: Wed, 11 Mar 2026 13:34:16 GMT
Content-Type: application/json
Content-Length: 455
Access-Control-Allow-Origin: *
Connection: close

[
  {
    "course": {
      "credits": 5,
      "id": 1,
      "title": "Computer Programming 101"
    },
    "id": 1,
    "student": {
      "email": "janel.perez@chmsu.edu.ph",
      "id": 1,
      "name": "Janel Perez"
    }
  },
  {
    "course": {
      "credits": 3,
      "id": 2,
      "title": "Accounting 101"
    },
    "id": 2,
    "student": {
      "email": "janel.perez@chmsu.edu.ph",
      "id": 1,
      "name": "Janel Perez"
    }
  }
]
```

Test	24. [EDGE CASE] Get enrollment — not found
curl command	`curl.exe -i "http://localhost:8003/enrollments/999"`
Expected	404 Not Found — {"error": "NOT_FOUND", "message": "Enrollment with id 999 does not exist"}

```
D:\Projects\microservices>curl -i "http://localhost:8003/enrollments/999"
HTTP/1.1 404 NOT FOUND
Server: Werkzeug/3.1.6 Python/3.14.3
Date: Wed, 11 Mar 2026 13:35:25 GMT
Content-Type: application/json
Content-Length: 81
Access-Control-Allow-Origin: *
Connection: close

{
  "error": "NOT_FOUND",
  "message": "Enrollment with id 999 does not exist"
}
```

Test	25. [EDGE CASE] Duplicate enrollment
curl command	`curl.exe -i -X POST "http://localhost:8003/enrollments" -H "Content-Type: application/json" -d "{\"student_id\":1,\"course_id\":1}"`
Expected	`409 Conflict — {"error": "DUPLICATE_ENROLLMENT", "message": "Student 1 is already enrolled in course 1"}`
Note	The same student-course pair from test 20 is submitted again. The service detects the duplicate before writing.

```
D:\Projects\microservices>curl -i -X POST "http://localhost:8003/enrollments" -H "Content-Type: application/json" -d "{\"student_id\":1,\"course_id\":1}"
HTTP/1.1 409 CONFLICT
Server: Werkzeug/3.1.6 Python/3.14.3
Date: Wed, 11 Mar 2026 13:36:20 GMT
Content-Type: application/json
Content-Length: 96
Access-Control-Allow-Origin: *
Connection: close

{
  "error": "DUPLICATE_ENROLLMENT",
  "message": "Student 1 is already enrolled in course 1"
}
```

Test	26. [EDGE CASE] Enroll with non-existent student
curl command	`curl.exe -i -X POST "http://localhost:8003/enrollments" -H "Content-Type: application/json" -d "{\"student_id\":999,\"course_id\":1}"`
Expected	`404 Not Found — {"error": "NOT_FOUND", "message": "Student with id 999 does not exist"}`
Note	The Enrollment Service validates the student ID against the Student Service before proceeding.

```
D:\Projects\microservices>curl -i -X POST "http://localhost:8003/enrollments" -H "Content-Type: application/json" -d "{\"student_id\":999,\"course_id\":1}"
HTTP/1.1 404 NOT FOUND
Server: Werkzeug/3.1.6 Python/3.14.3
Date: Wed, 11 Mar 2026 13:37:01 GMT
Content-Type: application/json
Content-Length: 78
Access-Control-Allow-Origin: *
Connection: close

{
  "error": "NOT_FOUND",
  "message": "Student with id 999 does not exist"
}
```

Test	27. [EDGE CASE] Enroll with non-existent course
curl command	`curl.exe -i -X POST "http://localhost:8003/enrollments" -H "Content-Type: application/json" -d "{\"student_id\":1,\"course_id\":999}"`
Expected	`404 Not Found — {"error": "NOT_FOUND", "message": "Course with id 999 does not exist"}`

```
D:\Projects\microservices>curl -i -X POST "http://localhost:8003/enrollments" -H "Content-Type: application/json" -d "{\"student_id\":1,\"course_id\":999}"
HTTP/1.1 404 NOT FOUND
Server: Werkzeug/3.1.6 Python/3.14.3
Date: Wed, 11 Mar 2026 13:37:36 GMT
Content-Type: application/json
Content-Length: 77
Access-Control-Allow-Origin: *
Connection: close

{
  "error": "NOT_FOUND",
  "message": "Course with id 999 does not exist"
}
```

Test	28. [EDGE CASE] Enroll with missing fields
curl command	`curl.exe -i -X POST "http://localhost:8003/enrollments" -H "Content-Type: application/json" -d "{\"student_id\":1}"`
Expected	`400 Bad Request — {"error": "MISSING_FIELDS", "message": "Both student_id and course_id are required"}`

```
D:\Projects\microservices>curl -i -X POST "http://localhost:8003/enrollments" -H "Content-Type: application/json" -d "{\"student_id\":1}"
HTTP/1.1 400 BAD REQUEST
Server: Werkzeug/3.1.6 Python/3.14.3
Date: Wed, 11 Mar 2026 13:38:20 GMT
Content-Type: application/json
Content-Length: 91
Access-Control-Allow-Origin: *
Connection: close

{
  "error": "MISSING_FIELDS",
  "message": "Both student_id and course_id are required"
}
```

Test	29. [EDGE CASE] Dependency failure — Student Service offline
curl command	`curl.exe -i -X POST "http://localhost:8003/enrollments" -H "Content-Type: application/json" -d "{\"student_id\":1,\"course_id\":1}"`
Expected	`503 Service Unavailable — {"error": "SERVICE_UNAVAILABLE", "message": "Student Service is currently unavailable"}`
Note	Run Stop-Job -Name StudentService in PowerShell before this test. Restart with Start-Job after.

```
D:\Projects\microservices>curl -i -X POST "http://localhost:8003/enrollments" -H "Content-Type: application/json" -d "{\"student_id\":1,\"course_id\":1}"
HTTP/1.1 503 SERVICE UNAVAILABLE
Server: Werkzeug/3.1.6 Python/3.14.3
Date: Wed, 11 Mar 2026 13:39:45 GMT
Content-Type: application/json
Content-Length: 94
Access-Control-Allow-Origin: *
Connection: close

{
  "error": "SERVICE_UNAVAILABLE",
  "message": "Student Service is currently unavailable"
}
```

All 29 test cases produced the expected HTTP status codes and JSON response bodies. The 400 Bad Request responses were consistently returned for any request missing required fields or providing values that failed validation, such as a credits value of zero. The error code in the response body differed between MISSING_FIELDS and INVALID_INPUT depending on whether the field was absent or present but unacceptable, providing more precise feedback to the client.

The 404 Not Found responses were correctly returned across all three services when a requested resource ID did not exist in the database. In the Enrollment Service, this extended to cross-service validation — a request referencing a student or course ID that did not exist in the respective upstream service returned 404 after the inter-service HTTP check, before any write was attempted.

The 409 Conflict response for duplicate enrollment was triggered correctly on the second submission of the same student-course pair. The check was performed at the application layer by querying the enrollment table before inserting, ensuring that the constraint was enforced even if the database did not have a unique index on the combination.

The 503 Service Unavailable response for dependency failure was verified by stopping the Student Service while the Enrollment Service remained running. A subsequent enrollment request correctly returned 503 with the SERVICE_UNAVAILABLE error code, confirming that the ConnectionError exception was caught and translated into the appropriate HTTP response. The Course and Student services remained independently accessible during this test, demonstrating fault isolation.

IV. LAB DISCUSSIONS / COMPUTATIONS

The test results confirmed that consistent error handling is achievable in a Flask-based microservice without a shared middleware layer. Each service independently enforces its validation rules and returns responses in the same structured format, which is sufficient for a system of this scale. In a larger production deployment, a shared error handling library or an API gateway that normalizes responses would reduce duplication and ensure consistency without relying on each service to implement the same convention independently.

The dependency failure test (test 29) illustrated a fundamental characteristic of distributed systems: a service that depends on another service inherits that service's availability as part of its own. The Enrollment Service cannot complete a write operation without first confirming that both the student and course exist, which means it is directly coupled to the availability of the Student and Course Services. This is not a design flaw — it is the correct behavior for a system that must maintain referential integrity across service boundaries. The key design decision is that the failure is surfaced immediately and clearly, rather than silently writing an orphaned record.

The absence of a cascade delete mechanism also became evident during testing. Deleting a student from the Student Service left any associated enrollment records in the Enrollment Service's database intact and unresolvable, since the enrollment's student reference would no longer match a valid resource. In a production system, this would be addressed by publishing a domain event from the Student Service on deletion and having the Enrollment Service subscribe to remove matching records asynchronously.

V. CONCLUSIONS

The edge case testing exercise demonstrated that robust microservice behavior requires explicit handling of every failure category — not just the happy path. Each of the five required edge case types (400, 404, 409, 503, 504) was successfully triggered and verified through curl, confirming that the Flask services behaved predictably under adverse conditions.

The exercise also reinforced that testing a distributed system requires a different mindset from testing a monolith. It is not sufficient to verify that a service works in isolation — the interactions between services, and the behavior of the system when those interactions fail, must be tested deliberately. The use of curl with the -i flag provided a direct, reproducible, and tool-independent way to verify HTTP-level behavior that is unambiguous and easy to include as committed evidence.

References

Abdelfattah, A. S., Cerny, T., Taibi, D., & Moha, N. (2023). A systematic mapping study on microservices testing. Journal of Systems and Software, 199, 111624. https://doi.org/10.1016/j.jss.2023.111624

Esposito, C., Castiglione, A., Cozzolino, G., & Chang, V. (2022). Fault-tolerant microservices for distributed and cloud computing environments. Future Generation Computer Systems, 135, 160–173. https://doi.org/10.1016/j.future.2022.04.023

Mozilla Developer Network. (2024). HTTP response status codes. Retrieved from https://developer.mozilla.org/en-US/docs/Web/HTTP/Status
Pallets Projects. (2024). Flask documentation (3.0.x). Retrieved from https://flask.palletsprojects.com/
Taibi, D., & Lenarduzzi, V. (2021). On the definition of microservice bad smells. IEEE Software, 38(3), 50–58. https://ieeexplore.ieee.org/document/8354414/

Waseem, M., Liang, P., Shahin, M., Di Salle, A., & Martini, A. (2022). Design, monitoring, and testing of microservices systems: The practitioners' perspective. Journal of Systems and Software, 187, 111196. https://arxiv.org/abs/2108.03384